

However, if in our number we don't have the first digit, the query will work:

```
forumdb=> select 0.123456789::numeric(10,10) as my_field;
      my_field
-----
0.1234567890
(1 row)
```

Now let's go back to the example of the previous paragraph, which provided an incorrect sum, and let's repeat it using the numeric type:

```
forumdb=> select sum(0.1::numeric(2,2)) from generate_series(1,10);
      sum
-----
1.00
(1 row)
```

As we can see, now the value of the sum is correct; so, the correct way to represent money is using a numeric data type.

Thus, we have learned all about the various numeric data types.

Character data type

The most used character data types in PostgreSQL are the following:

- `character(n)/char(n)` (fixed-length, blank-padded)
- `character varying(n)/varchar(n)` (variable length with a limit)
- `varchar/text` (variable unlimited length)

Now, we will look at some examples to see how PostgreSQL manages these kinds of data types.

Chars with fixed-length data types

We will check out how they work using the following example:

1. Let's start by creating a new test table:

```
forumdb=> create table new_tags (
pk integer not null primary key,
tag char(10)
);
CREATE TABLE
```

In the previous code, we created a new table named `new_tags` with a `char(10)` field name `tag`.

2. Now, let's add some records and see how PostgreSQL behaves:

```
forumbd=> insert into new_tags values (1,'first tag');
INSERT 0 1
forumbd=> insert into new_tags values (2,'tag');
INSERT 0 1
```

In order to continue with our analysis, we must introduce two new functions:

- `length(p)`: This counts the number of characters, where `p` is an input parameter and a string
- `octet_length(p)`: This counts the number of bytes, where `p` is an input parameter and a string

3. Let's execute the following query:

```
forumbd=> \x
Expanded display is on.
forumbd=> select pk,tag,length(tag),octet_length(tag),char_
length(tag) from new_tags;
-[ RECORD 1 ]+-----
pk          | 1
tag         | first tag
length      | 9
octet_length | 10
char_length | 9
-[ RECORD 2 ]+-----
pk          | 2
tag         | tag
length      | 3
octet_length | 10
char_length | 3
```

As we can see, the overall length of the space occupied internally by the field is always 10; this is true even if the number of characters entered is different. This happens because we have defined the field as `char(10)`, with a fixed length of 10, so even if we insert a string with a shorter length, the difference between 10 and the number of real characters of the string will be filled with blank characters.

Chars with variable length with a limit data types

In this section, we are going to repeat the same example that we used in the previous section, but this time, we'll use the `varchar(10)` data type for the tag field:

1. Let's recreate the `new_tags` table:

```
forumbd=> drop table if exists new_tags;
DROP TABLE

forumbd=> create table new_tags (
pk integer not null primary key,
tag varchar(10)
);
CREATE TABLE
```

2. Then, let's insert some data:

```
forumbd=> insert into new_tags values (1,'first tag');
INSERT 0 1

forumbd=> insert into new_tags values (2,'tag');
INSERT 0 1
```

3. Now, if we repeat the same query as before, we obtain the following:

```
forumbd=> \x
Expanded display is off.
forumbd=> select pk,tag,length(tag),octet_length(tag) from new_tags
;
pk | tag | length | octet_length
----+-----+-----+-----
 1 | first tag | 9 | 9
 2 | tag | 3 | 3
(2 rows)
```

As we can see, this time, the real internal size and the number of characters in the string are the same.

4. Now, let's try to insert a string longer than 10 characters and see what happens:

```
forumdb=> insert into new_tags values (3,'this sentence has more
than 10 characters');
ERROR:  value too long for type character varying(10)
```

PostgreSQL answers correctly with an error because the input string exceeds the dimension of the field.

Chars with a variable length without a limit data types

In this section, we will again use the same example as before, but this time, we'll use a text data type for the tag field.

Let's recreate the new_tags table and re-insert the same data that we inserted previously:

```
forumdb=> drop table if exists new_tags;
DROP TABLE

forumdb=> create table new_tags (
pk integer not null primary key,
tag text
);
CREATE TABLE

forumdb=> insert into new_tags values (1,'first tag'), (2,'tag'),(3,'this
sentence has more than 10 characters');
INSERT 0 3
```

This time, PostgreSQL correctly inserts all three records. This is because the text data type is a char data type with unlimited length, as we can see in the following query:

```
forumdb=> select pk,substring(tag from 0 for 20),length(tag),octet_
length(tag) from new_tags ;
 pk |      substring      | length | octet_length
----+-----+-----+-----
  1 | first tag           |      9 |           9
  2 | tag                 |      3 |           3
  3 | this sentence has m |     41 |          41
(3 rows)
```

In the preceding example, we can see that the text data type behaves exactly like the `varchar(n)` data type we saw earlier. The only difference between `text` and `varchar(n)` is that the `text` type has no size limit. It is important to note that in the preceding query, we used the `substring` function. The `substring` function takes a piece of the string starting from the `from` parameter for `n` characters; for example, if we write `substring(tag from 0 for 20)`, it means that we want the first 20 characters of the `tag` string as output.

With this, we have covered all the char data types.

Date/timestamp data types

In this section, we will talk about how to store dates and times in PostgreSQL. PostgreSQL supports both dates and times and the combination of date and time (timestamp). PostgreSQL manages hours both with time zone settings and without time zone settings, as described in the official documentation (<https://www.postgresql.org/docs/current/datatype-datetime.html>).



PostgreSQL supports the full set of SQL date and time types. Dates are counted according to the Gregorian calendar.

Date data types

Managing dates often becomes a puzzle for developers. This is because dates are represented differently depending on the country for which we have to store the data – for example, the American way is month/day/year, whereas the European format is day/month/year. PostgreSQL helps us by providing the necessary tools to best solve this problem, as seen here:

1. The first thing we have to do is to see how PostgreSQL internally stores dates. To do this, we have to perform the following query:

```
forumdb=> \x
Expanded display is on.
forumdb=> select * from pg_settings where name = 'DateStyle';
-[ RECORD 1 ]---+-----
--
name          | DateStyle
setting       | ISO, MDY
[..]
sourcefile    |
```

```
sourceline      |
pending_restart | f
```

First of all, let's take a look at the `pg_settings` view. Using the `pg_settings` view, we can view the parameters set in the `postgresql.conf` configuration file. In the preceding result, we can see that the configuration for displaying the date is `MDY` (month/day/year). If we want to change this parameter globally, we have to edit the `postgresql.conf` file.

2. On a Debian or Debian-based server, we can edit the file as follows:

```
root@pgdev:/# vim /etc/postgresql/16/main/postgresql.conf
```

3. Then, we have to modify the following section:

```
#Locale and Formatting

datestyle = 'iso, mdy'
```

4. After changing this parameter, in the query on `pg_settings`, the context parameter is 'user'; we just need to do a reload of the server. In this case, a restart is not necessary:

```
root@pgdev:/# service postgresql reload
[ ok ] Reloading postgresql configuration (via systemctl):
postgresql.service.
```

For further information about the `pg_settings` view, we suggest visiting <https://www.postgresql.org/docs/current/view-pg-settings.html>.

5. We have learned what the internal parameters for date display are, so now, let's look at how to insert, update, and display dates. If we know the value of the `date-style` parameter, the PostgreSQL way of converting a string into a date is as follows:

```
forumdb=> \x
Expanded display is off.
forumdb=> select '12-31-2020'::date;
       date
-----
2020-12-31
(1 row)
```

This way is simple but not particularly user-friendly. The best way to manage dates is by using some functions that PostgreSQL provides for us.

6. The first function that we'll talk about is the `to_date()` function. The `to_date()` function converts a given string into a date. The syntax of the `to_date()` function is as follows:

```
forumbd=> select to_date('31/12/2020','dd/mm/yyyy') ;
           to_date
           -----
           2020-12-31
(1 row)
```

The `to_date()` function accepts two string parameters. The first parameter contains the value that we want to convert into a date. The second parameter is the pattern of the date. The `to_date()` function returns a date value.

7. Now, let's go back to the posts table and execute this query:

```
forumbd=> \x
Expanded display is on.
forumbd=> select pk,title,created_on from posts;
-[ RECORD 1 ]-----
pk          | 5
title       | Indexing PostgreSQL
created_on  | 2023-01-23 15:21:55.747463+00
-[ RECORD 2 ]-----
pk          | 6
title       | Indexing Mysql
created_on  | 2023-01-23 15:22:02.38953+00
-[ RECORD 3 ]-----
pk          | 7
title       | A view of  Data types in C++
created_on  | 2023-01-23 15:26:21.367814+00
```

How is it possible that we have date/time combinations (timestamps) if nobody has ever entered these values into the table? It is possible because the posts table has been created as follows:

```
forumbd=> \d posts;
Table "public.posts"
Column          | Type          | [...] | Default
-----+-----+-----+-----
```

```

pk          | integer          |      | [...]
title       | text             |      |
[.....]
created_on  | timestamp with time zone |      | CURRENT_TIMESTAMP

```

As we can see, the `created_on` field has `CURRENT_TIMESTAMP` as the default value, which means that if no value has been inserted, the current timestamp of the server will be inserted. Suppose now that we want to display the date in a different format – for example, in the European format, `created_on: 03-01-2020`.

8. To reach this goal, we have to use another built-in function, the `to_char` function:

```

forumdb=> select pk,title,to_char(created_on,'dd-mm-yyyy') as
created_on
from posts;
-[ RECORD 1 ]-----
pk          | 5
title       | Indexing PostgreSQL
created_on  | 23-01-2023
-[ RECORD 2 ]-----
pk          | 6
title       | Indexing Mysql
created_on  | 23-01-2023
-[ RECORD 3 ]-----
pk          | 7
title       | A view of  Data types in C++
created_on  | 23-01-2023

```

As shown here, the `to_char()` function is the inverse of the `to_date()` function.

Timestamp data types

PostgreSQL can manage dates and times with a time zone and without a time zone. We can store both date and time using the `timestamp` data type. In PostgreSQL, there is a data type called `timestamp with time zone` to display date and time with a time zone, and a data type called `timestamp without time zone` to store date and time without a time zone.

Let's now go through some examples. First of all, let's create a new table:

```

forumdb=> create table new_posts as select pk,title,created_on::timestamp
with time zone as created_on_t, created_on::timestamp without time zone as

```

```
create_on_nt from posts;
SELECT 3
```

We have just created a new table called `new_posts` with the following structure:

```
forumdb=# \d new_posts;
Table "public.new_posts"
Column          | Type                               | [...]
-----+-----+-----
pk              | integer                           |
title           | text                              |
created_on_t    | timestamp with time zone          |
create_on_nt    | timestamp without time zone       |
```

This table now has the same values for the `create_on_t` (timestamp with time zone) field and for the `created_on_nt` (timestamp without time zone) field, as we can see here:

```
forumdb=> select * from new_posts ;
-[ RECORD 1 ]+-----
pk          | 5
title       | Indexing PostgreSQL
created_on_t | 2023-01-23 15:21:55.747463+00
create_on_nt | 2023-01-23 15:21:55.747463
-[ RECORD 2 ]+-----
pk          | 6
title       | Indexing Mysql
created_on_t | 2023-01-23 15:22:02.38953+00
create_on_nt | 2023-01-23 15:22:02.38953
-[ RECORD 3 ]+-----
pk          | 7
title       | A view of Data types in C++
created_on_t | 2023-01-23 15:26:21.367814+00
create_on_nt | 2023-01-23 15:26:21.367814
```

Now, let's introduce a PostgreSQL environment variable called the `timezone` variable. This variable tells us the current value of the time zone:

```
forumdb=> show timezone;
-[ RECORD 1 ]-----
TimeZone | Etc/UTC
```

In this server, the time zone is set to UTC; if we want to modify this value only on this session, we have to perform the following query:

```
forumdb=> set timezone='CET';
SET
```

Now, the time zone is set to CET:

```
forumdb=> show timezone;
-[ RECORD 1 ]-
TimeZone | CET
```

Now, if we execute the query that we performed previously again, we will see that the field with the time zone has changed its value:

```
forumdb=> select * from new_posts ;
-[ RECORD 1 ]+-----
pk          | 5
title       | Indexing PostgreSQL
created_on_t | 2023-01-23 16:21:55.747463+01
create_on_nt | 2023-01-23 15:21:55.747463
-[ RECORD 2 ]+-----
pk          | 6
title       | Indexing Mysql
created_on_t | 2023-01-23 16:22:02.38953+01
create_on_nt | 2023-01-23 15:22:02.38953
-[ RECORD 3 ]+-----
pk          | 7
title       | A view of Data types in C++
created_on_t | 2023-01-23 16:26:21.367814+01
create_on_nt | 2023-01-23 15:26:21.367814
```

This shows the difference between a timestamp with a time zone and a timestamp without a time zone. For further information on the topic of date and time, please refer to the official documentation at <https://www.postgresql.org/docs/current/datatype-datetime.html>.

The NoSQL data type

In this section, we will approach the NoSQL data types that are present in PostgreSQL. Since this book is not specifically focused on NoSQL, we will just take a quick look.